

Hermes Skill 自进化完整源码机制解析

0. 先区分两个“自进化”

Hermes 现在有两层容易混淆的机制：

1. Hermes Agent 主仓库里的运行时自学习闭环

也就是 Agent 在日常任务后，通过 Background Review 自动创建、修正、整理 `SKILL.md`。这是你问的核心机制。

2. `hermes-agent-self-evolution` 独立仓库里的 GEPA/DSPy 进化器

这是一个外部优化流水线，会读取 skill/prompt/tool、生成评测集、提出候选变体、跑评测和约束门控，最后通过 PR 合入。它不是 Hermes 主进程每轮对话自动运行的机制。该仓库 README 明确写的是用 DSPy + GEPA 优化 skills、tool descriptions、system prompts 和 code，并通过 tests、size limit、semantic preservation、PR review 等门控提交最优变体。

本文主要讲 **主仓库内置的 Skill 自进化闭环**。

1. 总体架构：Hermes Skill 自进化是三条闭环

从源码看，Hermes Skill 自进化不是“模型自己训练自己”，而是三条文件级闭环：

``text id="c1j8d9" 闭环一：任务中使用 Skill skills_list → skill_view → 加载 SKILL.md / 支持文件 → 执行任务 → 记录 usage

闭环二：任务后复盘 Skill run_conversation 结束 → TurnFinalizer 判断是否触发 review → spawn_background_review → fork 一个后台 Agent → 只开放 skill/memory 工具 → 判断是否 create / patch / write_file

闭环三：长期治理 Skill skill_usage 记录使用情况 → curator 定期判断 active / stale / archived → 可选 LLM consolidation → 合并碎片 Skill，归档低价值 Skill

因此，Hermes 的“自进化”本质是：

``text id="chhq8t"

经验发现 → 写入程序性记忆 → 下次检索加载 → 使用统计 → 生命周期治理

不是参数训练，而是 **procedural memory** 的文件系统自修改。

2. Skill 的存储形态：一个目录就是一个技能单元

源码 `tools/skills_tool.py` 把 Skill 定义为一个目录，其中核心文件是 `SKILL.md`，旁边可以有支持文件。模块注释明确说明：Skill 是带有 `SKILL.md` 的目录，支持 `references/`、`templates/`、`examples/`、`assets/` 等，并采用 Anthropic-style progressive disclosure，即渐进式披露。

典型结构是：

```
```text id="e6747s" ~/.hermes/skills/ |—— software-development/ |—— node-troubleshooting/
|—— SKILL.md |—— references/ |—— templates/ |—— scripts/ |—— assets/
```

源码中的 `'SKILLS_DIR'` 指向用户目录：

```
```text id="gysyys"
~/.hermes/skills
```

`skills_tool.py` 会扫描本地 skills 目录和外部 skill 目录，解析 YAML frontmatter，过滤禁用、平台不兼容、环境不满足的 Skill，然后返回精简元数据。

这说明 Hermes 不是把 Skill 当普通 prompt 字符串，而是把它当成一个 **可检索、可加载、可修改、可治理的文件系统对象**。

3. Skill 如何被使用： `skills_list` 与 `skill_view`

Hermes Skill 的运行时使用分两步。

3.1 `skills_list`：只列出摘要

`skills_list` 的作用是列出可用 Skill 的元信息，例如 name、description、category。它不会把所有 `SKILL.md` 全部塞进上下文。这一步负责“发现候选 Skill”。源码说明 `skills_list` 会创建本地 skill 目录、扫描可用技能、返回排序后的 metadata/categories，并提示使用 `skill_view` 加载完整内容。

3.2 `skill_view`：加载完整 Skill 或支持文件

当 Agent 判断某个 Skill 相关时，再调用 `skill_view(name)` 加载完整 `SKILL.md`。如果需要更长材料，再调用：

```
```text id="yl8xy1" skill_view(name, file_path="references/xxx.md") skill_view(name,
file_path="templates/xxx.md") skill_view(name, file_path="scripts/xxx.py")
```

源码里 `'skill_view'` 会做路径安全检查，禁止绝对路径和路径穿越；如果多个目录里有同名 Skill，还会拒绝猜测，要求使用更明确路径。

如果读取的是主 `'SKILL.md'`，它还会解析 linked files、metadata.tags、related\_skills、环境变量需

求、credential file 需求，并返回 readiness status。读取成功后会调用 `mark\_background\_review\_skill\_read`，这对后面的后台写入安全非常关键。

此外，`skill\_view` 成功后会 bump 使用计数：`bump\_view()` 和 `bump\_use()`，这些数据后面会被 curator 用来判断 Skill 是否活跃或陈旧。

所以，Skill 的使用链路是：

```
```text id="wycypf"
skills_list: 知道有哪些 Skill
    ↓
skill_view: 加载某个 Skill
    ↓
执行任务
    ↓
记录 usage
    ↓
为后续 curator 提供依据
```

4. 任务中如何产生“需要学习”的信号

Hermes 有两种学习入口：

```text id="9kf1y4" 入口 A：主 Agent 在任务中主动调用 skill\_manage 入口 B：任务结束后 Background Review 自动复盘

真正更重要的是入口 B。

源码 `conversation\_loop.py` 中，Agent 每发生一次工具调用迭代，如果 `skill\_manage` 工具存在，并且 `\_skill\_nudge\_interval > 0`，就会增加 `\_iters\_since\_skill` 计数。

任务结束后，`agent/turn\_finalizer.py` 会检查：

```
```python id="ri5yu0"
_should_review_skills = False

if (
    agent._skill_nudge_interval > 0
    and agent._iters_since_skill >= agent._skill_nudge_interval
    and "skill_manage" in agent.valid_tool_names
):
    _should_review_skills = True
    agent._iters_since_skill = 0
```

然后，如果本轮有最终回复、没有被中断，并且需要 memory review 或 skill review，就调用：

```
```python id="q7ytbq" agent._spawn_background_review( messages_snapshot=list(messages),
review_memory=_should_review_memory, review_skills=_should_review_skills,)
```

源码注释明确说，Background Review 在响应交付之后运行，因此不会和用户当前任务争模型注意力。

这意味着 Hermes 的学习不是每个 turn 都强制发生，而是：

```
```text id="8o22c8"
工具调用积累到一定程度
↓
本轮任务正常结束
↓
TurnFinalizer 判断需要 review
↓
后台线程启动学习复盘
```

GitHub issue 也记录了这一设计：_iters_since_skill 在 tool-calling iteration 中增加，达到 _skill_nudge_interval 后触发 _spawn_background_review(review_skills=True)，由后台 agent 使用 _SKILL_REVIEW_PROMPT 决定是否创建或更新 skill。

5. Background Review：自进化的核心

源码 agent/background_review.py 是 Skill 自进化最核心的文件。

模块注释明确说：每轮对话后，AIAgent.run_conversation 可能调用 spawn_background_review；它会启动一个 daemon thread，拿当前会话快照，fork 一个新的 AIAgent，让它回放对话并判断是否应该保存或更新 memory/skill。这个后台 fork 继承 runtime 配置，但工具白名单限制为 memory 和 skill management，非白名单工具会被拒绝。

也就是说，Background Review 是一个 **后台审稿 Agent**：

```
```text id="k01s9h" 主 Agent 完成任务 ↓ 复制 messages_snapshot ↓ 后台 fork 一个 review_agent ↓
review_agent 只看对话和工具轨迹 ↓ 判断是否需要更新 Skill ↓ 通过 skill_manage 写入
```

## 5.1 它不是继续做用户任务

源码里 \_run\_review\_in\_thread 创建 review\_agent 时会设置：

```
```text id="tu6z1m"
quiet_mode=True
max_iterations=16
skip_memory=True
_persist_disabled=True
_session_db=None
```

```
_session_json_enabled=False
compression_enabled=False
```

并且复用/继承主 agent 的 provider/model/platform，同时设置 tool whitelist，只允许 skills 和可选 memory 工具。

关键点：

```text id="v7oo74" 它不负责继续回答用户；它不把 review 对话写入真实 session；它不应调用普通任务工具；它只负责“是否沉淀经验”。

## ## 5.2 它使用什么提示词

Background Review 的 Skill 提示词非常激进，但有明确边界。源码中的 `\_SKILL\_REVIEW\_PROMPT` 要求 review agent 主动寻找可复用经验，并强调目标库是 **class-level skills**，不是一轮会话一个 Skill。它要求优先更新当前加载过的 Skill，其次更新已有 umbrella Skill，再考虑添加 references/templates/scripts，最后才创建新的类别级 Skill。

源码提示词中的学习信号包括：

```text id="a4gtb7"

1. 用户纠正了风格、工作流或方法
2. 任务中出现了非平凡技术、修复、绕过方案、debug pattern、tool pattern
3. 已加载的 Skill 错了、缺东西、过时了

同时它明确禁止保存：

```text id="t29ppw" 1. 环境依赖的临时故障 2. 对工具“坏了”的负面结论 3. transient errors 4. 一次性任务叙事

它要求保存的是 **fix / procedure / reusable workflow**，不是简单记录失败本身。

这就是 Hermes Skill 自进化的核心判断逻辑。

---

## # 6. Background Review 的写入流程

Background Review 本身不直接写文件，它只能调用 `skill\_manage`。

`tools/skill\_manager\_tool.py` 的模块注释直接说这是 Agent-managed skill creation/editing 工具，Skill 是 procedural memory；支持 `create`、`edit`、`patch`、`delete`、`write\_file`、`remove\_file` 六类动作。

完整写入链路如下：

```text id="d4g0nk"

Background Review 判断需要更新

```

↓
调用 skill_manage(action=...)
↓
skill_manage 做 background guard
↓
如果开启 write_approval, 则进入 pending
↓
如果允许直接写, 则进入具体 handler
↓
validate name / frontmatter / size / path
↓
atomic write / fuzzy patch
↓
security scan 可选
↓
清缓存、记录 usage/provenance

```

7. skill_manage 的六种写入动作

源码 schema 和工具说明中, `skill_manage` 支持六种动作:

| action | 源码作用 | 适用场景 |
|--------------------------|---|--------------------------------|
| <code>create</code> | 创建新 Skill 目录和 <code>SKILL.md</code> | 没有合适 Skill 时 |
| <code>patch</code> | 用 <code>old_string/new_string</code> 精确替换 | 小修小补, 首选 |
| <code>edit</code> | 整体重写 <code>SKILL.md</code> | 结构性重构 |
| <code>delete</code> | 删除或归档 Skill | 合并/废弃 |
| <code>write_file</code> | 写支持文件 | 长 references、templates、scripts |
| <code>remove_file</code> | 删除支持文件 | 清理过期材料 |

源码 schema 也强调: `patch` 是修复已有 Skill 的首选方式, `edit` 只适合大重写; 删除合并时应传 `absorbed_into`。

7.1 create

`_create_skill` 会做:

```

```text id="d8ty1h"
1. 校验 name/category/frontmatter
2. 检查内容大小
3. 检查是否已有同名 Skill
4. 创建目录
5. 原子写入 SKILL.md
6. 可选 security scan
7. 如果扫描失败则 rollback

```

源码还限制 `SKILL.md` 最大 100k 字符, 支持文件单文件最大 1MiB, Skill name 必须符合 lowercase/dot/underscore/hyphen 等规则。

## 7.2 `patch`

`\_patch\_skill` 是最常用的自进化写法。它会：

```
```text id="m71485"
```

1. 找到目标 Skill
2. 可指定 file_path patch 支持文件
3. 检查 old_string 是否唯一
4. 支持 fuzzy matching
5. 替换为 new_string
6. 如果 patch 的是 SKILL.md，则重新校验 frontmatter
7. 原子写回
8. 可选安全扫描
9. 失败则回滚

源码明确包含 find-replace、fuzzy matching、replace_all、frontmatter validation、rollback 等逻辑。

7.3 write_file

`_write_file` 用来把长材料拆到支持文件中，例如：

```
```text id="wto3sf" references/debugging.md templates/report-template.md scripts/helper.py assets/example.png
```

源码会检查路径是否合法、大小是否超限、目标 Skill 是否存在、背景写入是否被允许；如果文件已存在，Background Review 还必须先读过该文件。写入也是原子写入，并支持安全扫描回滚。

---

# 8. 自进化的关键安全机制一：read-before-write

这是源码里很重要的一点。

`skill\_manager\_tool.py` 中有 `mark\_background\_review\_skill\_read` 和 `\_background\_review\_read\_before\_write\_guard`。注释说明：Background Review 不能仅凭会话中的零散印象 patch/rewrite 一个 Skill；必须先通过 `skill\_view` 读过目标文件。`skill\_view` 会标记该 Skill 已被后台 review 读取，写入路径会检查这个标记。

也就是说：

```
```text id="ajjhxq"
```

后台 review 想改 SKILL.md

↓

必须先 skill_view(name)

↓

skill_view 标记“已读”

↓

skill_manage 才允许 patch/edit

如果想改支持文件：

```
```text id="aeh3po" 必须先 skill_view(name, file_path="references/xxx.md") 再 write_file / patch / remove_file
```

这解决的是“后台 Agent 凭空改文件”的问题。

---

#### # 9. 自进化的关键安全机制二：Background Write Guard

`\_background\_review\_write\_guard` 会拒绝后台 curator/review 修改某些 Skill：

```
```text id="awjhah"
```

1. pinned skills
2. external skill paths
3. protected built-ins
4. hub-installed skills
5. bundled skills

源码明确说 Background Review 写入这些目标会被拒绝。

这说明 Hermes 对 Skill 的所有权有区分：

```
```text id="mf2k3a" 用户前台明确创建/维护的 Skill：更像用户资产 后台自动创建的 Skill：可以被 curator 管理 bundled/hub/external Skill：默认不能被后台乱改
```

这也是为什么后面 `skill\_provenance.py` 要区分 foreground 和 background。

---

#### # 10. 自进化的关键安全机制三：write approval

`skill\_manage` 不是一定直接写文件。源码中 `\_apply\_skill\_write\_gate` 会检查是否开启写审批。如果 `write\_approval` 开启，那么 `create/edit/patch/delete/write\_file/remove\_file` 都会被 staged 到 pending，而不是立即落盘。审批通过后，`apply\_skill\_pending` 会在 bypass gate 的情况下重放这次写操作。

因此有两种模式：

```
```text id="gavk8g"
```

默认模式：

Background Review → skill_manage → 直接写 ~/.hermes/skills

审批模式：

Background Review → skill_manage → pending queue


```
用户 diff / approve / reject  
approve 后才真正写入
```

这就是防止 Agent 自己无限乱改 Skill 的人审闸门。

11. 自进化的关键安全机制四：guard_agent_created

源码注释写得很清楚：`skills.guard_agent_created` 是可选 security scanner，默认 False；启用后会扫描 Agent 创建的 Skill，发现 dangerous findings 时阻断或回滚。

它和 write approval 的区别是：

```
```text id="v97ja7" write_approval: 人类审批门 guard_agent_created: 机器安全扫描
```

两者可以同时开，也可以独立开。

---

# 12. 写入成功后发生什么

``skill_manage()`` dispatch 成功后，会做几件事：

```
```text id="nyxqeq"
```

1. 清除 skills system prompt cache
2. 更新 usage telemetry，例如 bump_patch / forget
3. 标记 agent-created provenance
4. 返回写入结果

源码明确说，成功写入后会 clear skills system prompt cache，并 bump telemetry；只有 `action=="create"` 且当前处于 `is_background_review()` 时，才标记为 agent-created。前台用户创建的 Skill 被视为 user-owned，不由 curator 自动管理。

这点很关键：

```
```text id="945e3w" 后台自动创建的 Skill：进入自治生命周期治理 用户前台创建的 Skill：不默认被自治 curator 管理
```

---

# 13. 使用统计：为什么 Skill 能被长期治理

``tools/skill_usage.py`` 负责 Skill telemetry。它把使用信息写到：

```
```text id="b5hhg2"
~/hermes/skills/.usage.json
```

源码说明，这个 sidecar 会记录 `skills_tool` 和 `skill_manager_tool` 的事件；curator 使用派生出的 activity timestamp 判断 Skill 活跃程度，并维护 active → stale → archived 生命周期。

记录内容包括：

```
```text id="pe7o7f" view 次数 use 次数 patch 次数 last_activity pinned agent-created state: active / stale / archived
```

这使得 Hermes 不只是“会写 Skill”，还会判断：

```
```text id="fo3x2h"
哪些 Skill 经常用？
哪些 Skill 从没用？
哪些 Skill 该合并？
哪些 Skill 该归档？
哪些 Skill 被用户 pin 住不能动？
```

14. Provenance：区分用户创建和后台创建

`tools/skill_provenance.py` 用 contextvar 区分写入来源。源码说明，只有 Background Review 创建的 Skill 会被标记为 agent-created，用于后续 curator 管理；它依赖 `AIAgent._memory_write_origin` 这样的上下文。

这避免了一个严重问题：

```
```text id="bscwrw" 不能因为某个 Skill 位于 ~/.hermes/skills/ 就默认它是 Agent 自动创建、可以随便归档或合并。
```

所以 Hermes 不是按文件位置判断 ownership，而是按 provenance 判断。

---

# 15. Curator：长期自进化治理器

如果说 Background Review 是“每轮任务后的学习”，那么 Curator 是“长期技能库维护”。

``agent/curator.py`` 的模块注释说，Curator 是 background skill maintenance orchestrator，会在 Agent idle 且距离上次运行超过 interval 时触发；职责包括自动生命周期转换、启动后台 AI Agent 去 pin/archive/consolidate/patch agent-created skills、持久化状态。它严格保证：只处理 agent-created skills，不自动硬删除，只归档；pinned 绕过；辅助 client 不污染 prompt cache。

默认参数包括：

```
```text id="b0ip59"
interval: 7 days
min_idle: 2 hours
stale_after: 30 days
archive_after: 90 days
consolidate: false by default
```

源码里明确列出了这些默认值。

15.1 自动生命周期转换

`apply_automatic_transitions` 会遍历每个 curator-managed skill:

```
```text id="oremyj" 1. pinned 跳过 2. cron 引用的 Skill 不自动转换 3. 根据 last_activity / created_at 计算活跃时间 4. active → stale 5. stale → archived
```

源码还给从未使用的新 Skill 留 grace period, 避免刚创建就被归档。

### ## 15.2 可选 LLM consolidation

Curator 默认不做 LLM 合并, 只做确定性的 inactivity prune; 如果开启 consolidation, 才会 spawn 一个 forked AIAgent 做更激进的技能整理。源码里 `run_curator_review` 会先做 automatic transitions, 然后如果 consolidation enabled 且有 candidates, 就启动 LLM review。

Curator 的 LLM prompt 很明确: 它不是被动审计, 而是 umbrella-building pass, 目标是把碎片技能合并成类别级技能; 不能碰 bundled/hub/external, 不能 delete 只能 archive, 跳过 pinned/protected built-ins, 不 prune cron=yes, 并且合并时要维护支持文件和相对链接。

### ## 15.3 Curator fork 的权限

Curator LLM review 也是 fork 一个 `AIAgent`, 设置 `max_iterations=9999`、`quiet`、`skip memory/context`, 并设置 `_memory_write_origin="background_review"`, 所以 `skill_manage` 的 background guards 仍然生效。

也就是说, Curator 不是绕过安全机制的超级用户。它仍然受 background write guard、read-before-write、pinned/protected 限制约束。

---

### # 16. Bundled Skill 同步: 系统内置 Skill 如何不被覆盖

Hermes 还有 `tools/skills_sync.py` 负责 bundled skills 同步。

源码说明: 它会把 repo 内的 `skills/` 同步到 `~/hermes/skills/`, 使用 `.bundled_manifest` 记录。新内置 Skill 会复制; 未修改的旧内置 Skill 可以更新; 用户自定义修改会跳过; 用户删除的 bundled Skill 会被尊重; 已经移除的 bundled Skill 会从 manifest 清理。

这解决了一个现实问题:

```
```text id="p8tnx3"
```

Hermes 升级时，如何更新官方 Skill，
但又不覆盖用户已经改过的 Skill？

答案是：

```
```text id="c8mwjl" manifest + 修改检测 + 用户删除尊重 + external index 防 shadowing
```

```

```

# 17. 完整执行链路：从任务到 Skill 更新

下面用完整时序图解释源码机制。

```
```text id="8qg4sf"
```

用户发起任务

↓

AIAgent.run_conversation

↓

PromptBuilder 构建系统提示、上下文、工具列表、Skill guidance

↓

Agent 可能调用 skills_list

↓

Agent 发现相关 Skill

↓

Agent 调用 skill_view 加载 SKILL.md

↓

skill_view 记录 view/use

↓

Agent 执行工具调用、完成任务

↓

conversation_loop 统计工具迭代 _iters_since_skill

↓

TurnFinalizer.finalize_turn

↓

如果达到 skill_nudge_interval 且 skill_manage 可用：
_should_review_skills = True

↓

先把最终回答返回给用户

↓

后台 spawn_background_review

↓

fork review_agent

↓

review_agent 读取会话快照

↓

只允许 skills/memory 工具

↓

```
根据 _SKILL_REVIEW_PROMPT 判断是否需要沉淀经验
↓
优先 skill_view 已有相关 Skill
↓
如果要改, 调用 skill_manage patch/edit/write_file/create
↓
skill_manage:
    background guard
    read-before-write
    write approval gate
    path/frontmatter/size validation
    atomic write
    optional security scan
    rollback on failure
↓
写入成功:
    clear skill prompt cache
    bump usage/provenance
↓
下次任务:
    skills_list / skill_view 能看到更新后的 Skill
↓
长期:
    curator 根据 usage 做 stale/archive/consolidation
```

这是 Hermes 主仓库中 Skill 自进化的完整闭环。

18. 举一个源码级例子

假设一次任务中, Agent 调用多个工具修复了一个 npm on Windows 的奇怪问题。用户最后还纠正它:

```text id="wwrxxk" 以后 Windows PowerShell 下不要直接用 ~/.xxx 路径, 要用 \$env:USERPROFILE 或 Expand-Path。

Hermes 内部可能这样运行:

## 第一步: 任务执行期间

Agent 可能已经加载了:

```text id="rqi42a"  
software-development/node-troubleshooting/SKILL.md

或者没有加载任何 Skill。

第二步：TurnFinalizer 判断触发 review

工具调用次数累计达到 `_skill_nudge_interval`，本轮正常结束，于是：

```
```text id="6l8lur" _should_review_skills = True _spawn_background_review(...)
```

这一触发逻辑在 `turn_finalizer.py` 中实现。

## 第三步：Background Review 复盘

后台 review prompt 发现：

```
```text id="scbqfp"
用户纠正了一个可复用工作流；
这是 Windows shell 路径处理的通用坑；
应该更新已有 software-development / powershell / node skill。
```

第四步：先读目标 Skill

如果它想 patch 已有 Skill，必须先：

```
```text id="uyoo5v" skill_view("node-troubleshooting")
```

因为 Background Review 有 read-before-write 限制。

## 第五步：patch Skill

然后调用：

```
```text id="g8ydgd"
skill_manage(
    action="patch",
    name="node-troubleshooting",
    old_string="...",
    new_string="..."
)
```

加入一条：

```
```markdown id="hihg17"
```

## Windows PowerShell Pitfalls

- Do not assume Unix-style `~/.config/path` expansion works in every shell. Prefer `$env:USERPROFILE`, `$HOME`, or explicit path expansion when giving commands for native PowerShell.

## 第六步：写入并记录

`skill\_manage` 校验 frontmatter、路径、大小，原子写回，清 cache，记录 patch telemetry。如果这个 Skill 是 Background Review 创建的，还会有 agent-created provenance。

## 第七步：下次复用

下次用户再问 Windows PowerShell 路径问题，Agent 通过 `skills\_list` 找到该 Skill，再通过 `skill\_view` 加载更新后的流程。

这就是一次完整的自进化。

---

# 19. Hermes Skill 自进化的几个关键设计点

## 19.1 它追求 class-level skill，不追求 one-task-one-skill

Background Review prompt 明确说目标库是 class-level skills，而不是 one-session-one-skill 的流水账。它要求优先更新已有 Skill 或 umbrella Skill，只有确实没有合适目标时才创建新 Skill。

这解决 Skill 库爆炸问题。

错误做法：

```
```text id="buc1gx"
fix-npm-error-2026-07-05
fix-npm-error-again
fix-windows-path-bug
```

正确做法：

```
```text id="t8vpun" software-development/windows-powershell/SKILL.md software-development/
node-troubleshooting/SKILL.md
```

## 19.2 它把长内容拆到支持文件

源码和 prompt 都鼓励把长 reference、template、script 放到支持目录，而不是全部塞进 `SKILL.md`。允许目录包括 `references/`、`templates/`、`scripts/`、`assets/`。

这让 `SKILL.md` 保持短小，长材料按需加载。

## 19.3 它把“失败”转化为“修复流程”

Background Review prompt 明确不要求记录 transient failure，而是记录 fix/workaround/procedure。

因此 Skill 不是错误日志，而是经验库。

## 19.4 它通过 provenance 防止误治理用户资产

只有后台创建的 Skill 会被标记为 agent-created; foreground 用户创建的 Skill 不默认由 curator 管理。

这避免系统把用户手写的 Skill 当自动垃圾清理掉。

## 19.5 它通过 read-before-write 防止凭空修改

后台 Agent 想改 Skill，必须先 `skill\_view`。这相当于一个最小证据门槛。

---

# 20. 和 Claude Code / OpenAI Skills 的区别

从机制上看，Hermes 的 Skill 更偏“自治维护的文件级程序性记忆”。

```text id="78bv3i"

Claude Code / OpenAI Skills:

- 强调 skill 作为可加载能力包；
- 主要是人写或外部安装；
- 是否自动更新取决于平台机制。

Hermes Skills:

- 除了加载，还把 Background Review、skill_manage、usage telemetry、provenance、curator、write approval 接进运行时；
- 因此有内置的自学习闭环。

它的特点不是“Skill 文件夹”本身，而是：

```text id="7rc8fq" 任务后复盘 + 自动 patch/create + 生命周期治理 + 权限门控

---

# 21. 源码机制的边界与风险

## 21.1 它不改变模型参数

所有变化发生在：

```text id="6sb01n"

- ~/.hermes/skills/
- ~/.hermes/skills/.usage.json
- pending skills queue
- curator state/report

不是权重更新。

21.2 它依赖后台 Agent 判断

Background Review 的判断仍然是 LLM 做的，所以可能：

```
```text id="d7wr5m" 过度学习 学到错误经验 把一次性经验泛化 更新不该更新的 Skill
```

Hermes 用 read-before-write、protected skill、pinned、write approval、guard scanner、curator provenance 来降低风险，但不能从理论上完全消除。

## 21.3 Curator consolidation 默认关闭

源码默认 `consolidate=false`，所以长期自动合并不是默认强开；默认更偏确定性的 inactivity lifecycle。

## 21.4 Background Review 用 “user role” 注入曾引发争议

GitHub issue 中有人指出，后台 review prompt 被作为 `role: "user"` 注入，会让并行 Agent 误认为是真实用户命令，从而修改 Skill。该 issue 还给出实际时间线和 workaround。

这说明 Hermes 的自进化虽然强，但 governance 设计非常关键。生产或个人长期使用时，应考虑开启：

```
```text id="x8cm9e"
skills.write_approval: true
skills.guard_agent_created: true
pin 关键 Skill
```

22. 如果你要复现 Hermes 的完整机制，至少需要这些模块

```
```text id="uw2g1j" 1. Skill 文件规范 - SKILL.md - YAML frontmatter - references/templates/scripts/assets
```

1. Skill 检索工具
2. skills\_list
3. skill\_view
4. linked file loading
5. usage bump
6. 主 Agent 触发器
7. 工具调用迭代计数
8. skill\_nudge\_interval
9. turn finalizer
10. 后台 review 触发

11. Background Review Agent
12. 会话快照
13. forked AI Agent
14. quiet mode
15. 不写真实 session
16. 只开放 memory/skill tools
17. Skill 写入工具
18. create
19. patch
20. edit
21. delete/archive
22. write\_file
23. remove\_file
24. 写入安全
25. read-before-write
26. path traversal 防护
27. frontmatter 校验
28. size limit
29. protected/bundled/hub/external guard
30. atomic write
31. rollback
32. 审批机制
33. pending queue
34. diff
35. approve/reject
36. approval 后 replay
37. 使用统计
38. .usage.json
39. view/use/patch counters
40. last\_activity
41. Provenance
42. foreground user-created
43. background agent-created
44. curator-managed 标记

45. Curator

46. active/stale/archived

47. inactivity pruning

48. optional LLM consolidation

49. no hard delete, archive only

50. pinned/cron/protected 跳过

---

# 23. 最凝练总结

Hermes Skill 自进化的完整机制可以压缩成下面这句话：

> Hermes 在任务执行中通过 `skills\_list/skill\_view` 使用 Skill，并记录 usage；任务结束后 `TurnFinalizer` 根据工具调用积累触发 `spawn\_background\_review`，后台 fork 一个只允许 memory/skill 工具的 review agent，按 `\_SKILL\_REVIEW\_PROMPT` 判断是否需要把用户纠正、成功修复、非平凡 workflow 写回 Skill；写入统一经过 `skill\_manage`，受到 read-before-write、路径校验、frontmatter 校验、write approval、security scanner、ownership guard 保护；写入后清缓存并记录 provenance/usage；长期由 curator 根据 active/stale/archived 生命周期对 agent-created Skill 做归档和可选合并。

更直白地说：

```text id="c0bg1k"

Hermes 的 Skill 自进化 =

任务经验 → 后台复盘 → 文件级 patch/create → 安全审批 → 下次加载 → 使用统计 → 长期治理

这才是 Hermes “Agent grows with you” 在源码里的真正实现。